

LLM-unterstütztes Kodieren von Zeitungsartikeln für quantitative Forschung

Dino Noah Wildi

1. Juli 2026



Programm

- ▶ Hintergrund: Warum LLMs?
- ▶ Möglichkeiten: Unsere Forschung
- ▶ Grundlagen: Wo fange ich an?
- ▶ Tools: Was kann ich benutzen?
- ▶ Limitationen: Was geht nicht?



Hintergrund



Grundproblem der quantitativen Textforschung

Wie übersetzen wir Texte in Zahlen?

Klassische Methoden:

- ▶ Von Hand: flexibel und teuer
- ▶ Einfache Wortanalysen: einfach, aber limitiert
- ▶ Klassische ML-Modelle (z.B. SVM, LDA): limitierter Kontext



Grundproblem der quantitativen Textforschung

Wie übersetzen wir Texte in Zahlen?

Klassische Methoden:

- ▶ Von Hand: flexibel und teuer
- ▶ Einfache Wortanalysen: einfach, aber limitiert
- ▶ Klassische ML-Modelle (z.B. SVM, LDA): limitierter Kontext

LLM-gestützte Methoden:

- ▶ BERT et al.: hohe Kontrolle, hoher Aufwand; spezialisiert
- ▶ **Generative LLMs:** einfache Nutzung, wenig Kontrolle



Vor- und Nachteile von LLMs

LLMs sind Werkzeuge, keine "silver bullets"

Vorteile:

- ▶ Hohe Kontextfähigkeit
- ▶ Komplexere Tasks möglich
- ▶ Benötigen wenig bis keine Trainingsdaten
- ▶ Anwendung relativ einfach

Nachteile:

- ▶ Unklare Auswirkungen von Anpassungen
- ▶ Grösserer Zufallsfaktor
- ▶ Zugriff auf HPC oder Cloud
- ▶ Anpassung auf spezifische Aufgaben schwieriger



Kernsätze



Was sind Kernsätze?

Ausdruck von Relationen in Texten:

Wer sagt oder macht **was** über **wen** und/oder **was**?



Was sind Kernsätze?

Ausdruck von Relationen in Texten:

Wer sagt oder macht **was** über **wen** und/oder **was**?

- ▶ Subjekt: Wer macht einen Claim?
- ▶ Objekt: Gegen wen ist der Claim gerichtet?
- ▶ Issue: Um welches Sachthema geht es?
- ▶ Richtung: Ist der Claim positiv oder negativ?



Beispiele

"Friedrich Merz fordert die Wiedereinführung der Wehrpflicht"

Friedrich Merz / Unterstützung / Einführung der Wehrpflicht



Beispiele

"Friedrich Merz fordert die Wiedereinführung der Wehrpflicht"

Friedrich Merz / Unterstützung / Einführung der Wehrpflicht

"Fridays for Future protestiert gegen die Pläne der Bundesregierung, das Verbrenner-Aus wieder aufzugeben"

FFF / Opposition / Bundesregierung; Issue: Verbrenner-Aus

FFF / Unterstützung / Verbrenner-Aus

Bundesregierung / Opposition / Verbrenner-Aus



Probleme mit Kernsätzen

Komplexe Aufgabe mit mehreren Challenges:

- ▶ Claims von Hintergrundinformation unterscheiden
- ▶ Information in Standardformat umwandeln
- ▶ Implizite Information erkennen

Hiwis sind teuer: 200 Arbeitsstunden pro Wahl
Aufgabe zu kompliziert für klassisches ML



LLM-Kodierung



Grundsätzlicher Workflow

1. Codebook in LLM-freundlichem Format definieren
2. Daten von Hand codieren: "gold standard" definieren
3. Prompts mit Hilfe von Codebook schreiben
4. Handcodierte Sätze mit LLM codieren
5. Ergebnisse vergleichen: Codieren LLMs richtig?
6. Prompts iterativ verbessern, ab 3) wiederholen



Codebook-Format

Simple relation

Definition: Code one core sentence if a grammatical sentence contains one relation between a subject and a target.

Clarification: Whenever a subject takes a stance towards a target, or takes an action against a target, there is a relation between them. The subject is always the actor that is taking the action or stance, even if they are not the grammatical subject of the sentence.

Negative Clarification: The subject needs to actively make a claim towards the target. If the claim is not a direct relation between the two, but only an interpretation or background information given by the writer, do not code.

Positive example: "Chancellor Merz continues to receive support from the SPD". There is a stance taken by the SPD towards the Chancellor. Code one core sentence.

Negative example: "The government has not created more jobs in green technology than there were when they took office". This is just background information given by a journalist. The sentence does not make clear whether the government has attempted to create green jobs or supports this goal. Do not code a core sentence.



Prompts

Strukturierter Systemprompt:

- ▶ Task kurz erklärt + erwartetes Wissen klarmachen
- ▶ Codebook als JSON-dictionaries
- ▶ Input- und Output-Formate beschreiben:
 - ▶ Input: Artikelausschnitt mit markiertem Satz
 - ▶ Output: JSON nach vordefiniertem Schema



Beispielprompt

In the worst case, one household was faced with a £250,000 bill for fixing the damage caused by poor installation. Thousands more were hit with bills ranging from £250 to £18,000 to rectify work so poorly done that in some cases it posed a direct threat to health from mould and damp. While the Serious Fraud Office is being urged to investigate, the government must now try to repair not just the damage to houses, but the reputation of a battered industry that is vital to efforts to reduce the cost of living and tackle the climate crisis. > **This week, the government set out its long-awaited “warm homes plan”, a £15bn investment to help reduce bills and increase clean energy.** < It will fund solar panels, batteries and low-carbon heat pumps, as well as home insulation measures.”

The Guardian, 25. Januar 2026



Beispieloutput

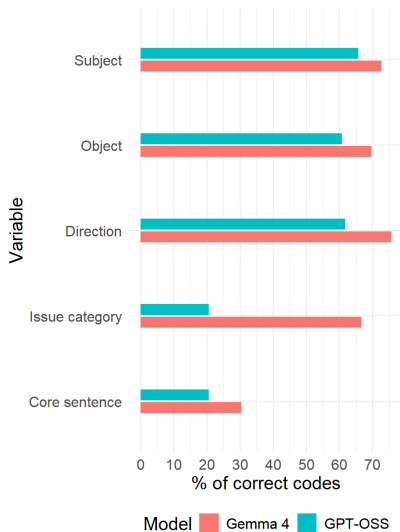
```
{ "sentence": "This week, the government set out its long-awaited "warm  
homes plan", a £15bn investment to help reduce bills and increase clean  
energy." },  
"core_sents": [{  
  "type": "actor-issue",  
  "subject": "the government",  
  "subject_organisation": "Labour Party",  
  "direction": "support",  
  "object": null,  
  "object_organisation": null,  
  "issue": "reduce bills",  
  "issue_cat": "COMPENSATION" } ]
```



Resultate



Resultate



Grundlagen



Modellauswahl

Grundsätzliche Fragen:

- ▶ Open-Source oder kommerziell?
 - ▶ Open-Source (oder open-weight) kann lokal genutzt werden
 - ▶ Open-weight ist reproduzierbar(er)
 - ▶ Open-source-Modelle können finetuned werden
 - ▶ Kommerzielle Modelle sind oft leistungsstärker



Modellauswahl

Grundsätzliche Fragen:

- ▶ Open-Source oder kommerziell?
- ▶ Modelltyp und -Familie
 - ▶ Modelle haben Stärken und Schwächen
 - ▶ Leaderboards helfen, aber passen nicht 1:1
 - ▶ Verschiedene Modellarchitekturen: MoE, Thinking, etc.



Modellauswahl

Grundsätzliche Fragen:

- ▶ Open-Source oder kommerziell?
- ▶ Modelltyp und -Familie
- ▶ Modellgrösse
 - ▶ Grösser ist tendenziell besser...
 - ▶ ... aber Familie > Grösse
 - ▶ Je grösser, desto mehr Rechenaufwand



Modellauswahl

Grundsätzliche Fragen:

- ▶ Open-Source oder kommerziell?
- ▶ Modelltyp und -Familie
- ▶ Modellgrösse
 - ▶ Grösser ist tendenziell besser...
 - ▶ ... aber Familie > Grösse
 - ▶ Je grösser, desto mehr Rechenaufwand

Best practice: Systematisches Testen (siehe z.B. McLaren et al. 2026)



Einstellungen

- ▶ **Temperature:** oft als "Kreativität" vereinfacht → je tiefer, desto höher werden wahrscheinliche Tokens gewichtet
- ▶ **Top-p / Top-k:** Wie viele Tokens kommen in Betracht; im Normalfall passt man eher die Temperature an
- ▶ **Random seed:** Festgelegter Seed = konsistenter Output
- ▶ **Context Window:** Wie viele Tokens behält das Modell im Speicher? Sollte immer höher sein als die Länge des Prompts.
- ▶ **max_tokens:** Maximum an Tokens, die generiert werden.



Promptarten

Userprompt: das, was wir alle von Chatbots kennen

Systemprompt: läuft im Hintergrund mit Typisches Setup:

Instruktionen im System Prompt, Input (z.B. Text) und evtl.
kleine Anmerkungen oder Reminders im User Prompt.

Vorteil: Prompts werden voneinander getrennt; Anweisungen und
Input beeinflussen sich nicht gegenseitig.



Strukturierte Outputs

Problem: LLMs generieren Text, wir wollen Daten

Lösung: Strukturierte Outputs in JSON-Form

Pydantic erlaubt uns, ein Schema festzulegen:

- ▶ Obligatorische und fakultative Variablen
- ▶ Auswahlmöglichkeiten für Variablen
- ▶ Wir können mehrere Klassen definieren



Tools und Skills



Python

- ▶ Standard für LLM-Anwendungen: Python
- ▶ Packages: numpy, pandas, pydantic, ollama
- ▶ Nötige Skills: Datenmanagement, LLM-Ansteuerung
- ▶ Einfache Beispielcodes auf GitHub (kommen noch)
- ▶ R-Anwendungen existieren, keine persönliche Erfahrung



Ollama

Ollama = Open-Source-Framework für lokale LLM-Anwendung

- ▶ Modelle suchen, downloaden und ansteuern
- ▶ LLMs lokal nutzen (bei genug Rechenpower)
- ▶ Ansteuerung via Python oder CMD
- ▶ Bietet auch Cloud-Dienste an (teils gratis)



bwHPC

Wir haben gratis Zugriff auf das **bwUniCluster** am KIT

- ▶ Anmeldung: <https://wiki.bwhpc.de/e/Registration/bwUniCluster>
- ▶ Zugriff auf Hochleistungsrechner und GPUs
- ▶ Steuerung via MobaXTerm oder CMD und Slurm (braucht grundlegende Skills, LLMs und Cluster-Team helfen beide)
- ▶ Dokumentation: https://wiki.bwhpc.de/e/Main_Page
- ▶ KIT bietet gratis Kurse an, Support ist sehr hilfreich



Probleme und Limitationen



LLM ist kein "künstlicher Hiwi"

"Künstliche Intelligenz" ist eine Fehlbezeichnung:

- ▶ LLMs "verstehen" Anweisungen nicht wie Menschen
- ▶ Spezifische Konzepte können falsch verstanden werden
- ▶ Kleine Änderungen im Wording können Auswirkungen haben

Lösung: möglichst präzise arbeiten, Prompts gut überprüfen,
Outputs immer vorsichtig validieren



Dokumentation und Reproduktion

- ▶ Modelle bleiben nicht immer verfügbar
- ▶ Kommerzielle Modelle werden manchmal still geändert
- ▶ Nicht-deterministischer Output $\Rightarrow$ unter Umständen mehrere Durchläufe nötig
- ▶ Setup kann Einfluss haben $\Rightarrow$ immer dokumentieren

